

EE 5239 Nonlinear Optimization

Lecture 13c: Sparse and Low Rank Optimization

Mingyi Hong, Jiaxiang Li
University of Minnesota

Outline

- 1 Review
- 2 Example
- 3 Review: Matrix Operations
- 4 Nuclear Norm Minimization

Announcement

- HW 5 due tonight (Nov 27th).
- HW 6 will be released today and due Dec 16th.
- Due date for the project report (and codes) will be the last day of the final exam (Dec 19.)

Last Time

- Mainly focused on sparse modeling
- Motivation behind sparse models

$$\|\mathbf{x}\|_1 = \sum_{k=1}^K |x_k|$$

- What types of regularization induces which types of sparsity
 - 1 $R(\mathbf{x}) = \|\mathbf{x}\|_1$, tend to produce $\mathbf{x}$ vector with a few nonzeros
 - 2 $R(\mathbf{x}) = \|\mathbf{x}\|_2$, tend to produce $\mathbf{x}$ vector with all zeros (only kink $\mathbf{x} = 0$)
 - 3 $R(\mathbf{x}) = \sum_{I=1}^G \|\mathbf{x}_{[I]}\|_2$, where $\mathbf{x}_{[I]}$ is a group of variables; tend to produce $\mathbf{x}$ vector with a few groups of nonzeros
 - 4 $R(\mathbf{x}) = \sum_{k=1}^{K-1} |x_k - x_{k+1}|$, tend to produce $\mathbf{x}$ vector with a few nonzeros adjacent differences

Last Time

- The Proximal Gradient Descent Method

$$\begin{aligned}\mathbf{x}^{r+1} &= \arg \min_{\mathbf{x}} \langle \nabla g(\mathbf{x}^r), \mathbf{x} - \mathbf{x}^r \rangle + \frac{L}{2} \|\mathbf{x} - \mathbf{x}^r\|^2 + \lambda R(\mathbf{x}) \\ &= \arg \min_{\mathbf{x}} \frac{L}{2} \|\mathbf{x} - (\mathbf{x}^r - \frac{1}{L} \nabla g(\mathbf{x}^r))\|^2 + \lambda R(\mathbf{x}) \\ &:= \text{prox}_{\lambda/L}^R \left(\mathbf{x}^r - \frac{1}{L} \nabla g(\mathbf{x}^r) \right)\end{aligned}$$

- For LASSO and Group LASSO, we see how the proximity operator can be computed in closed form
- Coordinate Descent is also a popular algorithm

This Time

- We discuss structured optimization for problems involving **matrices**
- How (and why) to encourage sparsity for matrices?
- How (and why) to encourage low-rankness in matrices?
- Good algorithms and applications

Application: Weight Decay

$$\min_W L_{\text{new}}(W) = L_{\text{original}}(W) + \lambda \|W\|_{\text{fro}}^2$$

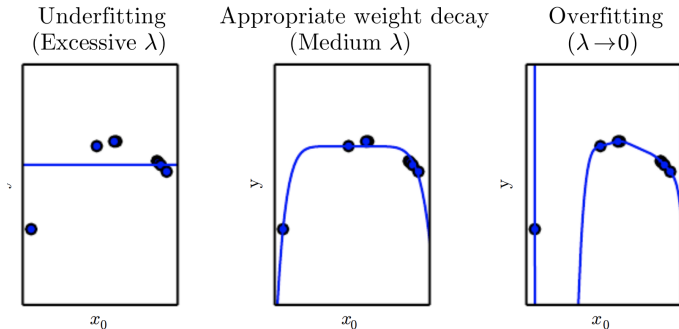
- W is the weight matrix
- Proximal operator of $R(\cdot) = \lambda \|\cdot\|_{\text{fro}}^2$ is

$$\text{prox}_R^\beta(X) = \frac{1}{1 + 2\beta\lambda} X$$

- Shrink the weight uniformly by a factor $1 + 2\beta$
- “Ridge regression”

Application: Weight Decay

- What do we gain by shrinking the weights?
- Prevent overfitting



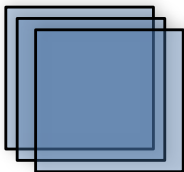
- Right now ℓ_1 regularization is not popular in deep learning. Why?

Application: Anomaly Detection

- **Objective.** Find unusual and rare events from a large amount of data
- Lots of applications, data analysis, imaging, cybersecurity
- **Ways.** Utilizing the sparsity of the “anomaly”

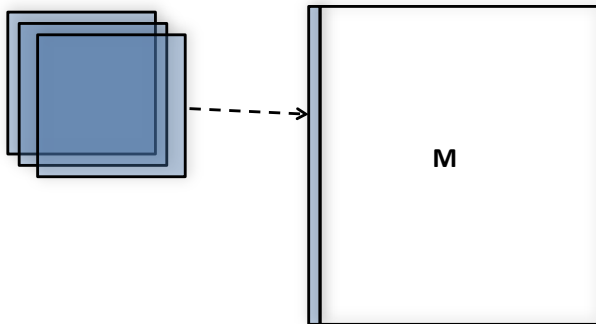
Application: Anomaly Detection

- An application on foreground separation from noisy surveillance video
- Suppose we have a sequence of video frames $\{\mathbf{M}^i\}_{i=1}^n$
- First we do preprocessing:



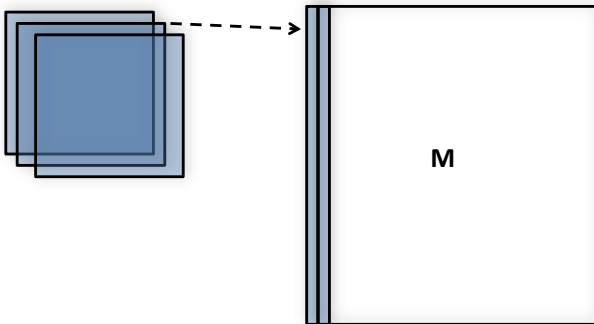
Application: Anomaly Detection

- An application on foreground separation from noisy surveillance video
- Suppose we have a sequence of video frames $\{\mathbf{M}^i\}_{i=1}^n$
- First we do preprocessing:



Application: Anomaly Detection

- An application on foreground separation from noisy surveillance video
- Suppose we have a sequence of video frames $\{\mathbf{M}^i\}_{i=1}^n$
- First we do preprocessing:



Application: Anomaly Detection

- We make each column of $\mathbf{M}$ becomes **one frame** of video
- **Objective:** Decompose it into three parts:
 - ① $\mathbf{L}$ is the background (**low rank**)
 - ② $\mathbf{S}$ is the moving foreground (**sparse**)
 - ③ $\mathbf{Z}$ is the noise (**small magnitude**)
- Why low rank matrix represents background? Changes slowly; consider a matrix with identical columns, the rank is **1**
- **Note:** No “label”, unsupervised learning!
- **Main Idea:** Promote these properties by penalizing using different costs: $(\min \text{Rank}(\mathbf{L}) + \text{Density}(\mathbf{S}) + \text{Magnitude}(\mathbf{Z}))$

Application: Anomaly Detection

$$\begin{array}{ll} \min & \|\mathbf{L}\|_* + \gamma_1 \|\mathbf{S}\|_1 + \gamma_2 \|\mathbf{Z}\|_F^2 \\ \text{subject to} & \mathbf{L} + \mathbf{S} + \mathbf{Z} = \mathbf{M} \end{array}$$

- 250×135 pixels per frame, 50 frames, over 1 million variables!!
- Similar idea applies to
 - 1 Analyzing power network security (intrusion detection)
 - 2 Data cleansing (getting rid of the data outliers)
 - 3 Network data flow with intrusion detection

Application: Anomaly Detection

$$\begin{aligned} \min_{\mathbf{L}, \mathbf{S}, \mathbf{M}} \quad & \|\mathbf{L}\|_* + \gamma_1 \|\mathbf{S}\|_1 + \gamma_2 \|\mathbf{Z}\|_F^2 \\ \text{subject to} \quad & \mathbf{L} + \mathbf{S} + \mathbf{Z} = \mathbf{M} \end{aligned}$$

$$\min_{\mathbf{L}, \mathbf{S}} \quad \|\mathbf{L}\|_* + \gamma_1 \|\mathbf{S}\|_1 + \gamma_2 \|\mathbf{M} - \mathbf{L} - \mathbf{S}\|_F^2$$

- 1 How to solve it?
- 2 Fix $\mathbf{L}$ update $\mathbf{S}$; fix $\mathbf{S}$ update $\mathbf{L}$

Application: Anomaly Detection

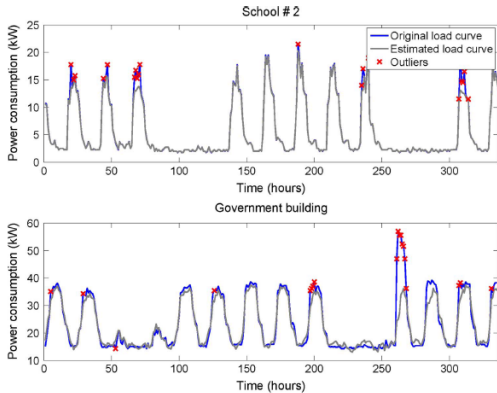


Figure 0.1: An Example in Power Data Cleansing and Imputation [Mateos and Giannakis 13]

Applications: Collaborative Filtering

- 1 Netflix predicts other “Movies You will love” based on past numeric ratings (1-5 stars)
- 2 Recommendations drive 60% of Netflix’s DVD rentals



Figure 0.2: [Mackey 2009]

Applications: Collaborative Filtering

- 1 Netflix Prize: Beat Netflix recommender System, using Netflix Data, Win \$ million
- 2 Data: 480,000 users, 18,000 movies, 100 mil observed ratings = only 1.1% of ratings observed



Figure 0.3: [Mackey 2009]

Example: Collaborative Filtering

- 1 **Question:** Will user i like movie j ?
- 2 **Collaborative Filtering:** Filtering information using techniques involving collaboration among multiple agents, view points, data sources.
- 3 **Matrix Completion:** Complete the matrix based on partially filled information

		movies									
users	1	2		1		4				5	
	2	5		4			?		1		3
	3		3		5		2				
	4	4		?		5		3		?	
	5			4		1	3			5	
	6			2				1	?		4
	7	1				5		5		4	
	8		2		?	5		?		4	
	9	3		3		1		5		2	1
	10	3				1			2		3
	11	4			5	1			3		
	12		3				3	?			5
	13	2	?		1	1					

Figure 0.4: [Scheinberg 2012]

Example: Collaborative Filtering

- Intuitively, collaborative filtering is to find a latent space

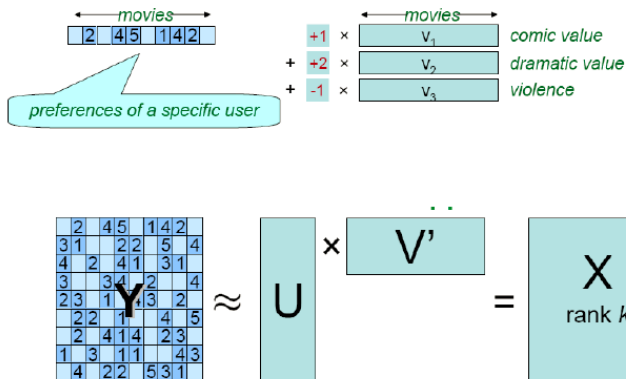


Figure 0.5: Formulation of Collaborative Filtering [Scheinberg 2012]

Example: Collaborative Filtering

- Suppose $\mathbf{M}$ is the observation, where the indices in the set $\mathcal{I}$ are those observed data points

$$\mathcal{I} := \{(i, j) \mid M_{i,j} \text{ is observed}\}$$

- We want to find a full matrix $\mathbf{X}$

Example: Collaborative Filtering

- Suppose that there are K **hidden** topics/factors representing each movie
- That is, a give movie i is represented by a vector

$$\mathbf{v}_i = [v_{i,1}; v_{i,2}; \dots; v_{i,K}]$$

- ① an index k represents a movie category
 - ② each $v_{i,k}$ represents the **strength of movie i in category k**
- A user j 's rating for movie i is modeled as

$$\mathbf{X}_{ij} = \mathbf{u}_i^T \mathbf{v}_j$$

where $\mathbf{u}_i = [u_{i,1}; \dots; u_{i,K}]$ is a vector representing **how much user i likes each category**

Example: Collaborative Filtering

- **Key Observation.** If we can learn the **users' preferences**, as well as the content of each movie, then we know $\mathbf{X}_{ij}$!
- We can formulate the Collaborative Filtering problem as

Decompose $\mathbf{X}$ into $\mathbf{U} \times \mathbf{V}^T$

where the following requirements are met

- 1 $\mathbf{X} \approx \mathbf{M}$
- 2 $\mathbf{X}$ has low rank
- 3 The original observations are all **consistent**

Example: Collaborative Filtering

- First suppose we know that the rank of X is K
- We can formulate the problem as a matrix factorization problem

$$\min_{\mathbf{U} \in \mathbb{R}^{m \times K} \mathbf{V} \in \mathbb{R}^{n \times K}} \sum_{(i,j) \in \mathcal{I}} \|\mathbf{U}_i \mathbf{V}_j^T - \mathbf{M}_{ij}\|^2$$

- Nonconvex formulation
- Algorithm: Alternating Least Squares (ALS)
 - 1 Fix $\mathbf{V}$, optimize $\mathbf{U}$, least square
 - 2 Fix $\mathbf{U}$, optimize $\mathbf{V}$, least square
- Basically a coordinate descent algorithm

Example: Collaborative Filtering

- What if we don't know the rank (or the order of the hidden factors?)
- More formally, we can write down the following optimization problem

$$\begin{aligned} \min_{\mathbf{X} \in \mathbb{R}^{m \times n}} \quad & \text{Rank}(\mathbf{X}) \\ \text{s. t.} \quad & X_{ij} = M_{ij}, (i, j) \in \mathcal{I} \end{aligned}$$

- **Question:** What is the rank of matrix $\mathbf{X}$? Easy to optimize?

Outline

- 1 Review
- 2 Example
- 3 Review: Matrix Operations
- 4 Nuclear Norm Minimization

Matrices

- ① Let $\mathbf{A}$ be a $m \times n$ matrix.
- ② Rank of $\mathbf{A}$ $\text{Rank}(\mathbf{A})$. Full rank matrix $\mathbf{A}$:
 $\text{Rank}(\mathbf{A}) = \min\{m, n\}$.
- ③ (Complex) Eigenvalue λ : $\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$ for some $\mathbf{x} \neq 0$.
- ④ Spectral radius: $\rho(\mathbf{A}) = \max_i \{|\lambda_i|\}$, λ_i is an eigenvalue of $\mathbf{A}$.
- ⑤ Eigen-decomposition of a symmetric matrix: $\mathbf{A} = \mathbf{P}'\mathbf{\Lambda}\mathbf{P}$; where $\mathbf{P}$ is orthonormal, $\mathbf{\Lambda}$ is diagonal and real.
- ⑥ Trace: $\text{Tr}[\mathbf{A}] = \sum_{i=1}^n \mathbf{A}_{ii} = \sum_{i=1}^n \lambda_i$

Matrices

- ① Singular value decomposition (SVD) (suppose $m \geq n$):

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}' \in \mathbb{R}^{m \times n},$$

where $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ rectangular diagonal matrix with diagonals $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$; $\mathbf{U} \in \mathbb{R}^{m \times m}$, $\mathbf{V} \in \mathbb{R}^{n \times n}$ orthonormal

- ② Let us use $\sigma(\mathbf{A})$ to denote the **vector** of singular values
- ③ What is the rank of $\mathbf{A}$?

$$\text{Rank}(\mathbf{A}) = \text{number of nonzero } \sigma_i \text{'s} = \|\sigma(\mathbf{A})\|_0$$

- ④ **Trace norm/nuclear norm** of a matrix $\mathbf{A}$ is

$$\|\mathbf{A}\|_* = \|\sigma(\mathbf{A})\|_1 = \sum_{i=1}^n \sigma_i = \text{Tr} \left(\sqrt{\mathbf{A}^T \times \mathbf{A}} \right)$$

Matrices and Norms

1 Norms:

$$\text{Frobenious Norm} : \|\mathbf{A}\|_F = \left(\sum_{i,j} |A_{ij}|^2 \right)^{1/2} = \left(\sum_i \sigma_i^2 \right)^{1/2} \quad (0.1)$$

$$\text{Nuclear Norm} : \|\mathbf{A}\|_* = \sum_i \sigma_i \quad (0.2)$$

$$\text{Matrix 2-norm} : \|\mathbf{A}\|_2 = \sup_{\mathbf{x} \neq 0} \frac{\|\mathbf{A}\mathbf{x}\|}{\|\mathbf{x}\|} = \max_i \sigma_i \quad (0.3)$$

- 2 Corresponds to **vector 2-norm** of singular values ($\|\cdot\|_2$), **vector 1-norm** of singular values ($\|\cdot\|_1$), **vector maximum norm** of singular values ($\|\cdot\|_\infty$)

- 3 Useful inequalities:

$$\|\mathbf{A}\mathbf{x}\| \leq \|\mathbf{A}\|_2 \|\mathbf{x}\|_2, \quad \|\mathbf{A}\|_* \geq \|\mathbf{A}\|_F \geq \|\mathbf{A}\|_2$$

Back to Collaborative Filtering

- Recall that the collaborative filtering problem has been formulated as

$$\begin{aligned} \min_{X \in \mathbb{R}^{m \times n}} \quad & \text{Rank}(\mathbf{X}) \\ \text{s. t.} \quad & X_{ij} = M_{ij}, (i, j) \in \mathcal{I} \end{aligned}$$

- This is difficult problem (NP-hard, as $\text{Rank}(\mathbf{X}) = \|\sigma(\mathbf{X})\|_0$)
- Practical Solution:** Use $\|\sigma(\mathbf{X})\|_1 = \|\mathbf{X}\|_*$ instead!

$$\begin{aligned} \min_{X \in \mathbb{R}^{m \times n}} \quad & \|\mathbf{X}\|_* \\ \text{s. t.} \quad & X_{ij} = M_{ij}, (i, j) \in \mathcal{I} \end{aligned}$$

Different Formulations

- The collaborative filtering (or the **matrix completion** problem) has several different formulations
- We have seen one formulation, which requires the observations M_{ij} to be exactly matched
- Other formulations includes

$$\begin{aligned} \min_{\mathbf{X} \in \mathbb{R}^{m \times n}} \quad & \|\mathbf{X}\|_* \\ \text{s. t.} \quad & \|\mathbf{X}_{\mathcal{I}} - \mathbf{M}_{\mathcal{I}}\|^2 \leq \epsilon \end{aligned} \quad \Leftrightarrow \quad \begin{aligned} \min_{\mathbf{X} \in \mathbb{R}^{m \times n}} \quad & \|\mathbf{X}\|_* + \lambda \|\mathbf{X}_{\mathcal{I}} - \mathbf{M}_{\mathcal{I}}\|^2 \end{aligned}$$

$$\begin{aligned} \min_{\mathbf{X} \in \mathbb{R}^{m \times n}} \quad & \frac{1}{2} \|\mathbf{X}_{\mathcal{I}} - \mathbf{M}_{\mathcal{I}}\|^2 \\ \text{s. t.} \quad & \|\mathbf{X}\|_* \leq R \end{aligned} \quad \Leftrightarrow \quad \begin{aligned} \min_{\mathbf{X} \in \mathbb{R}^{m \times n}} \quad & \lambda \|\mathbf{X}\|_* + \frac{1}{2} \|\mathbf{X}_{\mathcal{I}} - \mathbf{M}_{\mathcal{I}}\|^2 \end{aligned}$$

The Proximity Operator For Nuclear Norm

- Consider the following nuclear norm regularized problem

$$\min_{\mathbf{X}} g(\mathbf{X}) + \underbrace{\lambda \|\mathbf{X}\|_*}_{\text{nonsmooth regularizer } R(\mathbf{X})} \quad (0.1)$$

- Again we can solve it by proximal gradient descent method
- The key is to compute the proximity operator

$$\text{prox}_{\|\cdot\|_*}^{\gamma}(\mathbf{Y}) = \arg \min_{\mathbf{X} \in \mathbb{R}^{m \times n}} \gamma \|\mathbf{X}\|_* + \frac{1}{2} \|\mathbf{X} - \mathbf{Y}\|^2$$

- If this is easily done, then the proximal gradient algorithm is easy

$$\mathbf{X}^{r+1} = \text{prox}_{\|\cdot\|_*}^{\lambda/L} \left(\mathbf{X}^r - \frac{1}{L} \nabla g(\mathbf{X}^r) \right)$$

The Proximity Operator For Nuclear Norm

- Let's consider solving the problem

$$\min_{\mathbf{X} \in \mathbb{R}^{m \times n}} \gamma \|\mathbf{X}\|_* + \frac{1}{2} \|\mathbf{X} - \mathbf{Y}\|^2$$

- The optimal solution is very simple
 - First compute SVD of $\mathbf{Y}$: $\mathbf{Y} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$
 - Then define a diagonal matrix $\mathbf{\Sigma}(\gamma)$ as

$$\mathbf{\Sigma}(\gamma)_{ii} = \max(\mathbf{\Sigma}_{ii} - \gamma, 0)$$

- Set $\mathbf{X}^* = \mathbf{U}\mathbf{\Sigma}(\gamma)\mathbf{V}^T$
- Perform “soft-thresholding” on the set of singular values!
 - Compare with the ℓ_1 minimization problem we seen before (whose solution is $\text{rm Soft}(\mathbf{b}, \gamma)$)

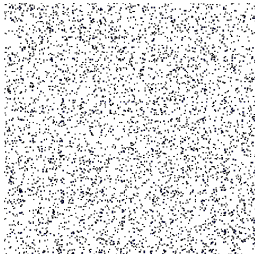
$$\min_x \gamma \|\mathbf{x}\|_1 + \frac{1}{2} \|\mathbf{x} - \mathbf{b}\|^2$$

An Example

- Consider the collaborative filtering problem

$$\min \sum_{(ij) \in \mathcal{I}} (X_{ij} - M_{ij})^2 + \lambda \|\mathbf{X}\|_* \quad (0.2)$$

- Requirement** the observed entries to be approximately identified $X_{ij} \approx M_{ij}$, the rest is free
- Nuclear norm added to enforce low rank in $\mathbf{X}$
- Specifically let $m = n = 500$, $|\mathcal{I}| = 5000$



Convergence

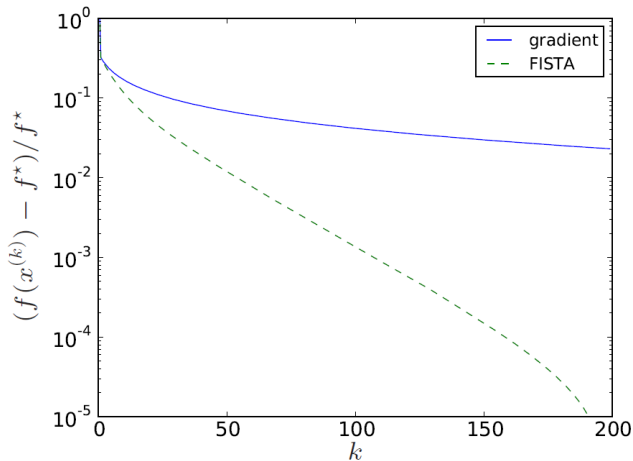


Figure 0.1: Convergence Speed (Proximal Gradient vs. its accelerated version) [Ghaoui10]

The Obtained Rank

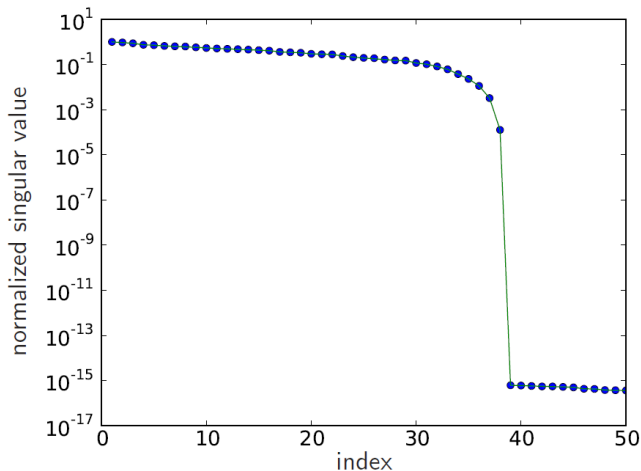


Figure 0.2: Optimal $\mathbf{X}$ has rank 38; relative error in specified entries is 9%
[Ghaoui10]